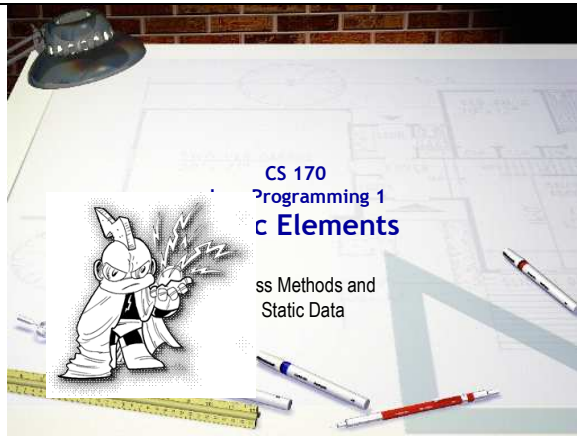




Slide 1

CS 170

Java Programming 1 Static Elements

Duration: 00:00:45

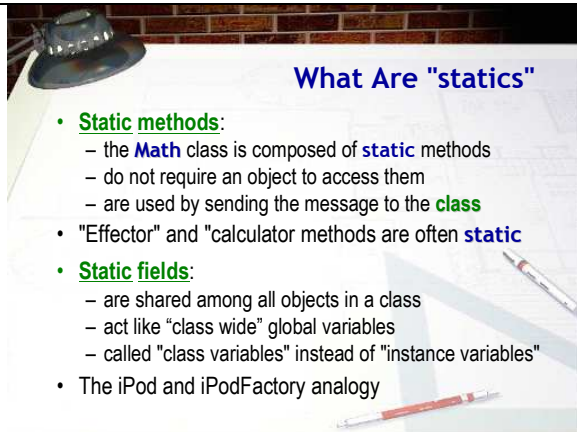
Advance mode: Auto

Notes:

Hi Folks, welcome to the Java Programming 1 lecture on Static Elements.

This week we're going to cover just some of the topics from Chapter 8 in your textbook. In this lecture, we'll cover the material from Section 8.6, Static Methods and Section 8.7, Static Fields, along with Advanced Topic 8.4, Static Imports.

So, what exactly do we mean when we talk about static elements? A static element is one that persists as long as your program is running. In other words, unlike an object instance, it doesn't come into existence when an object is constructed. In Java, we have both static fields and static methods.



Slide 2

What Are "statics"

Duration: 00:02:44

Advance mode: Auto

Notes:

You actually first met static methods back in Chapter 4, when you were introduced to the Math class. Static methods don't require you to create a Math object, like you'd done for other classes; Instead, you called these static methods by using the name of the class as the method receiver. In this lecture, you'll learn how to add static methods to the classes that you design and how to implement static fields to accompany them. Often, effector methods, that just carry out some kind of action, and calculator methods, like those found in the Math class, are designed as static methods, since they have no need to access or modify an object's state.

So, what are static fields? Static fields are shared fields. You can think of them as semi-global variables; they're not accessible everywhere in your program, but they are shared among all the objects in the class that contains them.

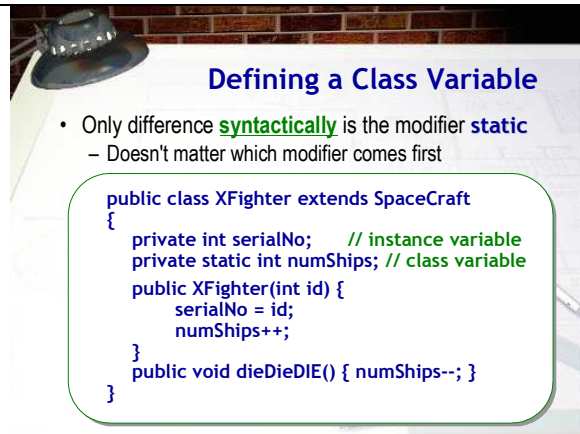
A static is often called a class variable, as opposed to an instance variable. You can think of it as belonging to the class as a whole, rather than to any individual object. When you create an object from a class, that object gets a copy of each of the instance variables defined by the class. The same object, however, does not get a copy of the class variables; there is only one copy of each class variable, "living" inside the class.

Here's an analogy that I read in a discussion group, and that I thought was very useful. Suppose that you're building a piece of software that simulates an iPod factory. Each iPod would have certain data associated with it, and one of those pieces of data would be the individual iPod serial number. In your program, serialNumber would be an instance variable because each iPod would have a unique value and that value should be attached to the individual iPod object.

In the iPod constructor, you'd want to initialize the serial number field, but you'd also have to make sure that no two iPods had the same serial number. How would you do that?

The easiest way is for the iPod class itself to keep a "master serial number" that individual iPod objects can access, and that gets updated to "the next number" whenever the current one is used by an iPod object.

If you think of a class as a "factory" for producing objects, then the serialNumber belongs to the individual iPod, while the masterSerialNumber data belongs to the iPodFactory.



Defining a Class Variable

- Only difference **syntactically** is the modifier **static**
 - Doesn't matter which modifier comes first

```
public class XFighter extends SpaceCraft
{
    private int serialNo;    // instance variable
    private static int numShips; // class variable
    public XFighter(int id) {
        serialNo = id;
        numShips++;
    }
    public void dieDieDIE() { numShips--; }
}
```

Slide 3

Defining a Class Variable

Duration: 00:02:07

Advance mode: Auto

Notes:

To define a class variable, you just add the keyword static as one of the modifiers appearing before the variable type. There are no rules about the order of modifiers, as long as it comes before the type.

Here's an example that illustrates shows both a class variable and an instance variable. (This is from a "SpaceWars" type game.) Notice serialNo is an instance variable and numShips is a class variable because it was declared with the keyword static. The class variable numShips is an example of a population counter. In the "SpaceWars" game, each XFighter object would get its own serialNo value, but all of the XFighters would share the single copy of numShips.

The constructor shown here is responsible for giving each ship a serial number, and incrementing the total number of ships. When the ship is destroyed, the dieDieDIE() method is called, which decreases the total number of XFighter objects in existence, hopefully with some kind of colorful explosion.

While the counter numShips works fine for keeping track of the number of XFighter objects, it has one nagging problem: only XFighter objects can find out how many XFighter exist! How does the UniversalOverlord object find out if he (or she!) needs to create more XFighters?

A regular method won't work in this case. Assume that you had a method named getNumShips() that returned the value of numShips. To use the method, the UniversalOverlord would have to create, and destroy, a dummy XFighter, just to find out how many fighters exist. Then it would have to subtract one from the result just to eliminate the dummy variable we created. (Not to mention putting up with the pyrotechnics generated by dieDieDIE(). This is where we need the static method.

Defining static Methods

- To **define** a **static** method, use the **static** modifier
 - Only** access **static** fields or other **static** methods
 - You **cannot** access **any** instance variables

```
public static int getNumShips()
{
    // cannot call any instance methods in class
    // may call other static methods
    // may access static fields like numShips
    // cannot access serialNo from this method
    return numShips;
}
```

Notes:

To define a static method, just add the word static as one of the modifiers; it must go before the return type, but does not have to go before the access specifier (public or private).

Inside the body of the static method, you cannot refer to any instance variables. You may create local variables, and you can refer to static fields, but not instance variables. Since the getNumShips() method only refers to the static variable numShips, the code it contains is fine.

Slide 4

Defining static Methods

Duration: 00:00:43

Advance mode: Auto

Calling static Methods

- Users **call** a **static** method by using the **class** name:

```
int ships = XFighter.getNumShips();
```

- No need to create an XFighter object first.
- In Java, users **may** call a **static** method using an instance of the **class** though

```
XFighter xf = new SFighter(52);
int ships = xf.getNumShips(); // very uncommon
```

Notes:

To use a static method, you use the name of the class, instead of the name of an object, when calling the method. Here's the code that UniversalOverlord would use to call the method:

Note that we need neither create, nor destroy, an XFighter object, to find out how many exist.

In Java, users may call a static method using an instance of the class like the example shown here. This is very uncommon, though, and you should really avoid doing it.

Write static methods whenever you need a method that will work without first creating an object.

Slide 5

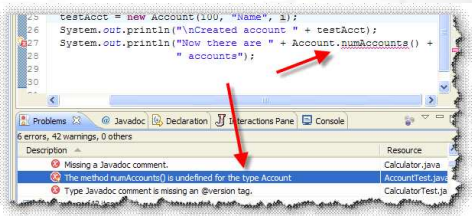
Calling static Methods

Duration: 00:00:46

Advance mode: Auto

Express Yourself

- Exercise 1:** Using static variables and static methods
 - Download and import Eclipse project **ic17**
 - Open **Account.java** and **AccountTest.java**
 - Try to run **AccountTest**. Does it run? Why not?



Notes:

OK. Now that you know the syntax for creating a static field and a static method, let's give it a try. Create this week's IC exercise document and then download and import the Eclipse project ic17. (You'll use Import Projects into Workspace. You can find ic17.zip on the class Web page this week.)

Once you have the project open, then open Account.java and AccountTest.java. Then, try to run AccountTest. Does it run? No! As you can see here, the numAccounts method is not defined inside the Account class. Also, as you might notice, numAccounts() is a static method, since it is called using the Account class and not the object named testAcct.

Your job is to implement the numAccounts() static method in the Account class. To do that you'll need to add a static variable to the Account class and increment it every time an object is created.

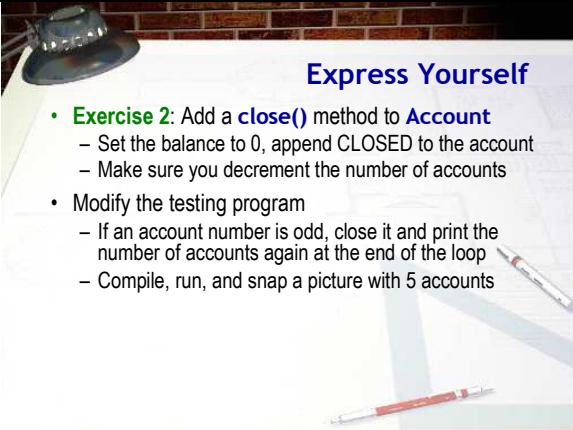
When you're done, run AccountTest once again and enter 5

Slide 6

Express Yourself

Duration: 00:01:33

Advance mode: Auto

	when asked how many accounts to create. For Exercise 1, shoot me a screenshot showing your code and the output on the console window.
 Slide 7  Express Yourself Duration: 00:00:53 Advance mode: Auto • Exercise 2: Add a <code>close()</code> method to <code>Account</code> – Set the balance to 0, append CLOSED to the account – Make sure you decrement the number of accounts • Modify the testing program – If an account number is odd, close it and print the number of accounts again at the end of the loop – Compile, run, and snap a picture with 5 accounts	Notes: Now, let's try to do something similar to <code>dieDieDIE()</code> in the <code>XFighter</code> class. Add a <code>close()</code> method to the <code>Account</code> class. This is a non-static, normal instance method. Inside the <code>close</code> method, set the balance to 0 and append the word "CLOSED" on to the end of the account name. Whenever an account is closed, you'll need to decrement the number of active accounts. Now, modify the testing program. Inside the loop that creates accounts, if an account number is odd, then close the account after you've created it. After the end of the loop, print out the number of existing accounts one final time. Compile, run, and shoot me a final screenshot.